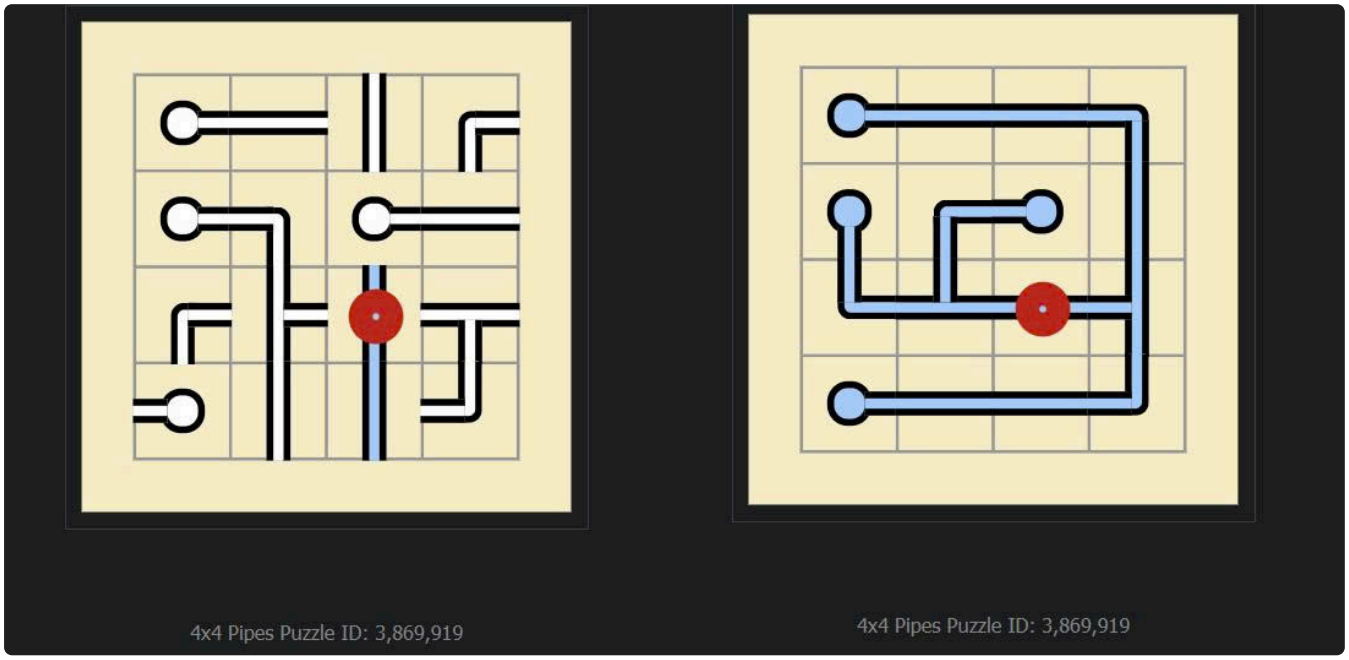
Phiếu bàn giao kỹ thuật – Pipes – TV2

Oct 26, 2026 · @Tuanh

1. Luật chơi

Pipes (puzzle-pipes.com) là lưới vuông, mỗi ô chứa một đoạn ống thuộc một trong bốn loại: đầu cụt (1 lỗ), ống thẳng (2 lỗ đối nhau), ống góc (2 lỗ kề nhau), chữ T (3 lỗ). Người chơi chỉ được **xoay** từng ô 90°, không được đổi chỗ hay đổi loại ống. Bài được giải khi mọi đầu ống khớp với đầu ống của ô bên cạnh, không đầu nào chĩa ra ngoài biên, tất cả các ô nối thành **một mạng duy nhất** và mạng **không có vòng kín**. Nói theo đồ thị: các ô là đỉnh, các chỗ nối là cạnh, và lời giải là một **cây khung** phủ hết lưới.



Trái: đề 01 (4×4, Puzzle ID 3,869,919) với góc xoay ban đầu; các ống màu xanh là phần đang nối với nguồn (chấm đỏ). Phải: lời giải, mọi ô nối thành một mạng duy nhất, không đầu hở và không vòng. Ảnh chụp từ puzzle-pipes.com.

2. Dữ liệu đầu vào

Nhóm dùng 5 đề lấy từ puzzle-pipes.com, kích thước tăng dần từ 4×4 đến 15×15, mỗi đề có kèm lời giải đúng.

Định dạng file input (`inputs/pipes/01.txt` … `05.txt`):

- Các dòng bắt đầu bằng `#` là chú thích: nguồn, Puzzle ID, quy ước mã, và dòng `# source: i j` ghi vị trí chấm đỏ trên web (chỉ để vẽ, không ảnh hưởng luật).
- Dòng đầu tiên không phải chú thích: `rows cols`.

- Tiếp theo là `rows` dòng, mỗi dòng `cols` ký tự hex cách nhau bởi dấu cách. Mỗi ký tự là bitmask các phía có lỗ của ô: Trên = 1, Phải = 2, Dưới = 4, Trái = 8. Ví dụ `A` = 2+8 = ống thẳng ngang, `C` = 8+4 = góc trái–dưới, `D` = 1+8+4 = chữ T.

Ví dụ đề 01:

```
# Pipes 4x4 - input
# Nguon: puzzle-pipes.com, Puzzle ID 3,869,919
# Ma o: tong cac huong co lo, Tren=1 Phai=2 Duoi=4 Trai=8 (hex)
# source (cham do, dong cot tinh tu 0): 2 2
4 4
2 A 5 6
2 C 2 A
6 7 5 E
8 5 5 9
```

Danh sách đề (nguồn: puzzle-pipes.com, lấy ngày 24/10/2026):

File	Kích thước	Puzzle ID	Số ô	Đầu ống hở ban đầu (h1)
01.txt	4×4	3,869,919	16	16
02.txt	5×5	8,335,297	25	32
03.txt	7×7	7,075,572	49	52
04.txt	10×10	723,397	100	110
05.txt	15×15	5,804,275	225	228

File lời giải nằm ở `inputs/pipes/solutions/01.txt` … `05.txt`, cùng định dạng. Chúng nằm trong thư mục con để benchmark (đọc `inputs/pipes/*.txt`) không chạy nhầm file lời giải như một đề.

Cách kiểm tra lời giải là đúng:

1. File lời giải phải qua `goal_test`: không còn đầu hở, đúng 1 cụm liên thông, 0 vòng.
2. Mỗi ô của đề phải là một phép xoay của ô tương ứng trong lời giải (cùng loại ống).
3. Hai điều trên được kiểm tra tự động trong `tests/test_pipes.py`. Ngoài ra, lời giải DFS tìm được trùng từng ô với file lời giải ở cả 5 đề.

3. Biểu diễn trạng thái

Trạng thái là một **tuple các số nguyên 0–15**, mỗi số là bitmask lỗ của một ô sau khi xoay, xếp theo thứ tự hàng → cột (ô (i, j) ở vị trí i·cols + j). Cùng kiểu dữ liệu này dùng cho hai giải thuật với hai cách hiểu khác nhau:

	DFS	Hill Climbing
Ý nghĩa	Trạng thái **dở dang**: góc xoay đã chốt cho k ô đầu tiên	Trạng thái **đầy đủ**: mọi ô đều có góc xoay, có thể đang sai
Độ dài tuple	0 đến rows·cols	Luôn bằng rows·cols
Trạng thái đầu (đề 01)	`()`	`random_state(rng)`, với seed 0: `4AAC / 1685 / 6D5D / 1AA9` (h1 = 6)
Trạng thái đích (đề 01)	`2AAC / 4685 / 3BAD / 2AA9` (đủ 16 ô)	`2AAC / 4685 / 3BAD / 2AA9`

(Viết gọn theo hex, dấu `/` ngăn các hàng; trong code là tuple 16 số, ví dụ `(2, 10, 10, 12, 4, 6, 8, 5, 3, 11, 10, 13, 2, 10, 10, 9)`.)

Vì sao chọn cách này:

- **Bitmask:** xoay 90° chỉ là xoay vòng 4 bit, `((m << 1) | (m >> 3)) & 15`; kiểm tra "ô có lỗ hướng X" là một phép AND; hai ô khớp nhau khi `bool(A & Dưới) == bool(B & Trên)`. Mã này cũng trùng với file input và với ký hiệu GUI vẽ.
- **Tuple:** hashable và bất biến, đúng quy ước của `core/problem.py` (DFS của TV1 lưu trạng thái vào `set`).
- **DFS gán dần từng ô:** không gian trở thành **cây** (mỗi trạng thái chỉ đến được bằng một đường, không bao giờ lặp), và mỗi lần chốt một ô là kiểm tra được ngay ô đó có hỏng không, nên cắt nhánh rất sớm. Nếu để DFS xoay tự do trên lưới đầy đủ thì trạng thái lặp lại (xoay 4 lần về chỗ cũ) và không cắt nhánh được.
- **Hill Climbing cần trạng thái đầy đủ:** giải thuật không lưu đường đi, chỉ đứng ở một điểm và so h của các điểm láng giềng, nên mỗi điểm phải là một lưới hoàn chỉnh.
- Hai cách hiểu dùng chung `goal_test`, `h`, `encode`: `goal_test` chỉ đúng khi tuple đủ số ô; `h` coi ô chưa gán là ô trống; `encode` in ô chưa gán là `.`.

4. Không gian trạng thái

Theo format slide chương 2 (State space search: trạng thái, trạng thái đầu, trạng thái đích, luật chuyển), cho DFS:

- **Trạng thái đầu:** `()`, chưa chốt góc xoay cho ô nào.

- **Tập hành động** `actions(s)`: các góc xoay khác nhau của ô kế tiếp (ô thứ `len(s)`). Ống thẳng có 2 góc; đầu cụt, góc, chữ T có 4 góc.
- **Hàm chuyển trạng thái** `result(s, a)` = `s + (a,)`: chốt góc `a` cho ô kế tiếp, trả về tuple mới.
- **Kiểm tra hợp lệ** `is_valid(s)`: gọi ngay sau khi sinh nút con, chỉ xét ô vừa gán (ô trên và ô trái đã gán, ô dưới và ô phải chưa). Loại nếu:
  1. **(a) Chĩa ra biên:** ô có lỗ hướng ra ngoài lưới.
  2. **(b) Lệch với ô trên hoặc ô trái:** một bên có lỗ hướng vào bên kia mà bên kia không có lỗ quay lại.
  3. **(c) Tạo vòng kín:** ô mới nối cả lên trên lẫn sang trái, mà hai ô đó vốn đã thông nhau qua đường khác.
  4. **(d) Tạo cụm kín:** cụm chứa ô mới không còn lỗ nào hướng tới ô chưa gán trong khi lưới chưa đủ, nên cụm này không bao giờ nối được với phần còn lại.
- **Trạng thái đích** `goal_test(s)`: đủ rows·cols ô, số đầu hở = 0, số cụm liên thông = 1, số vòng = 0 (đếm bằng union-find; số vòng = số cạnh − số ô + số cụm).
- **Chi phí mỗi bước:** 1 (mặc định `step_cost`). Mọi lời giải cùng độ sâu rows·cols nên chi phí không ảnh hưởng kết quả.

Với Hill Climbing (trạng thái đầy đủ), "luật chuyển" là `neighbors(s)`: xoay 1 ô, hoặc xoay đồng thời 2 ô kề nhau (chi tiết ở mục 6); đích vẫn là `goal_test`.

Kích thước không gian. Số cách xoay cả lưới = tích số góc xoay của từng ô. Đề 01 có 10 ô 4 góc và 6 ống thẳng 2 góc, nên có $4^{10} \times 2^{6} = 67.108.864$ cách. Con số này tăng theo hàm mũ với số ô, trong khi DFS có cắt nhánh chỉ mở rộng vài trăm nút:

Đề	Kích thước	Số cách xoay cả lưới	Nút DFS thực sự mở rộng
01	4×4	$\approx 6{,}7 \times 10^{7}$	27
02	5×5	$\approx 3 \times 10^{13}$	31
03	7×7	$\approx 5 \times 10^{27}$	66
04	10×10	$\approx 6 \times 10^{54}$	174
05	15×15	$\approx 2{,}5 \times 10^{123}$	823

Duyệt hết 10^{123} trạng thái là không thể. DFS chạy được vì ràng buộc (b) gần như ép chết góc xoay của mỗi ô ngay khi gán, nên cây tìm kiếm thực tế rất hẹp. Số nút lấy từ `results/benchmark.csv`.

5. Heuristic

Nhóm dùng **h1 = số đầu ống hở** cho Hill Climbing; h2 có thêm thông tin toàn mạng nhưng không giải được nhiều đề hơn và không tính tăng dần được.

Công thức.

h1(n) = số đầu ống chĩa ra biên hoặc chĩa vào ô không có lỗ quay lại

h2(n) = h1(n) + (C(n) − 1) + L(n), với **L(n) = E(n) − V + C(n)**

- h1: đếm từng lỗ của từng ô; lỗ nào không được nối thì +1. h1 = 0 nghĩa là mọi lỗ đều khớp.
- C(n): số cụm liên thông (chỉ tính các cạnh khớp hai đầu). C − 1 = số lần còn phải nối cụm.
- L(n): số vòng độc lập; E = số cạnh khớp, V = số ô. Một rừng C cây có V − C cạnh, mỗi cạnh thừa tạo thêm một vòng.
- h2 = 0 khi và chỉ khi là lời giải. h1 = 0 chưa chắc là lời giải (mọi lỗ khớp nhưng còn tách cụm hoặc có vòng), vì vậy giải thuật luôn gọi `goal_test` thay vì dừng khi h = 0.

Ví dụ tính tay trên trạng thái xuất phát của Hill Climbing ở đề 01 (seed 0):

```
4 A A C
1 6 8 5
6 D 5 D
1 A A 9
```

Ô	Mã	Lỗ	Chĩa vào	Vì sao hở
(0,1)	A	Trái	(0,0) = 4	4 chỉ có lỗ Dưới, không có Phải
(2,1)	D	Dưới	(3,1) = A	A không có lỗ Trên
(2,2)	5	Trên	(1,2) = 8	8 không có lỗ Dưới
(2,2)	5	Dưới	(3,2) = A	A không có lỗ Trên
(2,3)	D	Trái	(2,2) = 5	5 không có lỗ Phải
(3,1)	A	Trái	(3,0) = 1	1 không có lỗ Phải

Vậy h1 = 6. Trạng thái này có C = 4 cụm và L = 0 vòng, nên h2 = 6 + (4 − 1) + 0 = 9. Cả hai số trùng với giá trị code trả về.

Admissible? Theo slide chương 3, h(n) là chi phí thật của đường rẻ nhất từ n tới đích, h*(n) là ước lượng của nó; admissible khi h*(n) ⩽ h(n) với mọi n. Coi mỗi bước là đổi góc một ô, thì **h1 không admissible**: một lần xoay có thể sửa tới 8 đầu hở (4 lỗ của chính ô đó và 4 lỗ của hàng xóm chĩa vào nó). Phản ví dụ: ở đề 01 ban đầu, h1 = 16 nhưng chỉ 13 ô khác lời giải, tức

chỉ cần 13 bước. h1/8 thì admissible nhưng không đổi thứ tự các láng giềng nên không có ích gì thêm. Hill Climbing không có g(n) và không đòi hỏi lời giải tối ưu, nên không cần h admissible; chỉ cần h giảm dần khi tiến gần đích.

Local hay global. Theo ví dụ Block World trong slide chương 3: h1 là **local heuristic** (mỗi ô chỉ xét với hàng xóm sát cạnh); h2 là **global heuristic** (thêm số cụm và số vòng, là thông tin về cấu trúc cả mạng).

So sánh h1 và h2. Đo bằng engine thử của TV2 trên máy cloud, 500 restart, trước khi có tính h tăng dần (không phải số liệu chính thức):

Đề	Cấu hình	h1	h2
03 (7×7)	không đi ngang	giải được, 9,5 s	giải được, 28,1 s
03 (7×7)	đi ngang 50	giải được, 0,09 s	giải được, 0,30 s
04 (10×10)	không đi ngang	thất bại, h tốt nhất 6	timeout 300 s, h tốt nhất 9
04 (10×10)	đi ngang 50	giải được, 45,1 s, 13 restart	giải được, 31,7 s, 3 restart
05 (15×15)	đi ngang 50	timeout, h tốt nhất 4	timeout, h tốt nhất 5

h1 được chọn vì ba lý do. Hai heuristic giải được cùng một tập đề. h1 tính rẻ hơn vì không cần union-find, nên đa số trường hợp nhanh hơn khoảng 3 lần (riêng 10×10 có đi ngang thì h2 cần ít restart hơn). Quan trọng nhất, h1 là local nên **tính tăng dần được** (mục 6): xoay 1–2 ô chỉ làm đổi số đầu hở quanh các ô đó. Tối ưu này nhanh hơn khoảng 5 lần và là điều giúp giải được đề 15×15. h2 không làm vậy được, vì xoay một ô có thể làm đổi số cụm của cả lưới.

6. Chi tiết giải thuật heuristic: Hill Climbing

Pipes dùng **Steepest-Ascent Hill Climbing** có random restart và đi ngang (`core/search.py` của TV1), đi theo hướng giảm h1 về 0. Tham số chính thức: **100 lần restart, 50 bước đi ngang, seed 0**.

Vòng lặp. Mỗi lần thử bắt đầu từ `random_state(rng)`. Mỗi bước sinh **toàn bộ** láng giềng, tính h cho từng cái và chọn cái có h nhỏ nhất (nhiều cái bằng nhau thì chọn ngẫu nhiên). Nếu h giảm thì đi tiếp. Nếu h bằng hiện tại và còn lượt thì đi ngang. Còn lại là kẹt ở cực tiểu địa phương hoặc vùng bằng phẳng, và giải thuật restart. Dừng khi `goal_test` đúng.

Trạng thái xuất phát (`random_state`). Mỗi ô chỉ chọn trong *miền* của nó, tức các góc xoay không chĩa ra biên (ô góc đề 01 chỉ còn 1 góc). Có hai chế độ:

- `greedy` (mặc định): đi hàng → cột, mỗi ô chọn ngẫu nhiên trong các góc khớp với ô trên và ô trái đã đặt; không có góc nào khớp thì chọn bất kỳ trong miền. Kết quả là lưới "gần đúng", chỉ sai rải rác. Ở đề 01 với seed 0, lưới xuất phát đúng 10/16 ô và có h1 = 6 (so với

16 của đề gốc). Đây là ý "getting it started right in the right general neighborhood" trong slide chương 3.

- `uniform`: mỗi ô chọn ngẫu nhiên trong miền, dùng để so sánh.

Seed. `hill_climbing` tạo `rng = random.Random(seed)` và truyền vào `random_state(rng)`. Mọi lựa chọn ngẫu nhiên (điểm xuất phát, chọn giữa các láng giềng bằng nhau) đều lấy từ `rng` này, không dùng `random` toàn cục. Vì vậy cùng seed thì ra cùng kết quả: hai lần chạy benchmark cho cùng số nút và số restart, và `core.check` xác nhận `random_state()` lặp lại được.

Láng giềng (`neighbors`) gồm hai loại:

1. **Xoay 1 ô** sang một góc khác trong miền.
2. **Xoay đồng thời 2 ô kề nhau**, chỉ với các cặp năm quanh một đầu ống đang hở.

Cần loại 2 vì tình huống **ridge** trong slide chương 3 ("two moves executed serially may increase the height"): khi hai ô cạnh nhau cùng sai và phải nối với nhau, xoay riêng từng ô chỉ chuyển chỗ hở từ bên này sang bên kia, h không giảm. Chỉ xét cặp quanh đầu hở để số láng giềng không bùng nổ. Nếu xét mọi cặp kề nhau, đề 15×15 có 420 cặp × tối đa 9 tổ hợp mỗi bước. Đề 01 ở trạng thái xuất phát có 45 láng giềng.

Ví dụ chạy thật trên đề 01 (seed 0, đi ngang 50): giải được ngay lần thử đầu, không restart.

Bước	Số láng giềng	Loại bước	Ô được xoay	h
Xuất phát				6
1	45	xuống dốc (loại 2)	(2,1) D→B và (2,2) 5→A	2
2	18	đi ngang	(0,0) 4→2 và (1,0) 1→2	2
3	44	đi ngang	(1,0) 2→1	2
4	26	đi ngang	(1,0) 1→4 và (2,0) 6→3	2
5	21	xuống dốc	(3,0) 1→2	0, `goal_test` đúng

Bước 1 là ví dụ của ridge: chỉ xoay cùng lúc hai ô mới giảm h từ 6 xuống 2. Bước 2–4 là vùng bằng phẳng: không láng giềng nào có h < 2, nên Hill Climbing thuần sẽ dừng ở đây.

Restart và đi ngang. Theo `results/benchmark_variants.csv` (100 restart):

- **Không đi ngang:** giải được 3/5 đề. 01, 02, 03 cần lần lượt 2, 21, 63 lần restart; 04 và 05 thất bại sau 100 lần, h tốt nhất còn 8 và 22.
- **Đi ngang 50:** giải được 5/5. 01–03 không cần restart, 04 cần 3 lần, 05 cần **82 lần**.

Khác biệt này cho thấy vấn đề chính của Pipes là vùng băng phẳng (rất nhiều láng giềng cùng h), chứ không chỉ đỉnh cục bộ. Đi ngang có giới hạn 50 bước liên tiếp để không lang thang mãi trên vùng phẳng.

Tính h tăng dần (TV1 tối ưu trong `neighbors`). Bản đầu tính lại h1 trên cả lưới cho từng láng giềng, mà đề 15×15 có vài nghìn láng giềng mỗi bước. Bản mới dựa vào việc lúc sinh láng giềng đã biết ô nào vừa xoay, nên chỉ đếm lại đầu hở của các ô đó và hàng xóm sát cạnh:

h1(láng giềng) = h1(hiện tại) − số đầu hở trong vùng (trước khi xoay) + số đầu hở trong vùng (sau khi xoay), trong đó vùng = các ô vừa xoay và hàng xóm sát cạnh.

Giá trị được lưu vào `_hcache` để `h()` trả về ngay. Kiểm chứng trên 86.471 láng giềng ngẫu nhiên của cả 5 đề: khớp 100% với cách tính cũ. Hill Climbing ra cùng lời giải, cùng số restart, cùng số nút, nhưng nhanh hơn khoảng 5 lần (đề 04: 4,4 s xuống 0,84 s trên máy thử). Nhờ vậy đề 15×15 giải được trong giới hạn 300 s.

7. Kiểm thử

`tests/test_pipes.py` có **20 test, pass toàn bộ** (`python -m unittest tests.test_pipes -v`). `core.check` báo **0 lỗi** trên cả 5 đề.

Unit test:

Nhóm	Kiểm tra gì	Bắt lỗi gì
Ô ống (2 test)	`rotate` xoay đúng chiều; ống thẳng có 2 góc, loại khác 4 góc	Sai nền tảng mã hóa
File input (2 test)	Có ≥ 5 đề; mỗi ô đề là phép xoay của ô lời giải	Chép đề sai
Lời giải đúng phải qua (2 test)	Lời giải qua `goal_test`, h = 0; **mọi tiền tố của lời giải đều qua `is_valid`**	`is_valid` cắt nhầm nhánh đúng, DFS không bao giờ tìm ra lời giải
Trạng thái sai phải bị loại (7 test)	Đề gốc không phải đích; chĩa ra biên; lệch ô trái; lệch ô trên; vòng kín (lưới 2×2 bốn ống góc); cụm kín (hai đầu cụt quay vào nhau); lưới khớp hết lỗ nhưng tách 2 cụm cho h1 = 0, h2 = 1	Mỗi test ứng với một điều kiện (a)–(d)

Nhóm	Kiểm tra gì	Bắt lỗi gì
Hill Climbing (3 test)	Láng giềng khác đúng 1 ô, hoặc 2 ô kề nhau, không chĩa ra biên; `random_state` cùng seed cho cùng kết quả; `encode` đúng kích thước và ký hiệu	Sai cấu trúc láng giềng, benchmark không lặp lại
Chạy với engine của TV1 (4 test)	Registry nạp được bài; DFS giải cả 5 đề; Hill Climbing giải 3 đề đầu; tracer ghi được JSON	Lệch interface với `core/`

Kết quả `python -m core.check pipes inputs/pipes/01.txt`:

```
1. Đọc input
  [ OK ] load_problem() trả về PipesProblem 4x4
2. Trạng thái đầu và các hàm DFS/A*
  [ OK ] initial_state() hashable
  [ OK ] Đi ngẫu nhiên 1 bước: result() không sưả state cũ, mọi state
hashable
3. Các hàm Hill Climbing
  [ OK ] random_state() hashable và lặp lại được với cùng seed
  [ OK ] neighbors() trả về 43 láng giềng hashable
4. Chạy thử giải thuật (tối đa 20 s mỗi giải thuật)
  [ OK ] dfs: tìm thấy lời giải trong 0.000 s, 27 nút mở rộng
  [ OK ] hc: tìm thấy lời giải trong 0.001 s, 5 nút mở rộng
Kết quả: 0 lỗi, 0 chú ý.
```

Với đề 02–05, `core.check` cũng báo 0 lỗi, DFS giải được, nhưng có 1 chú ý: Hill Climbing không tìm ra lời giải. Nguyên nhân là `core.check` chạy Hill Climbing với tham số mặc định (20 restart, không đi ngang), đúng hiện tượng vùng băng phẳng ở mục 6. Với tham số chính thức (100 restart, đi ngang 50), Hill Climbing giải đủ 5/5 trong `results/benchmark.csv`.

8. Khó khăn và cách xử lý

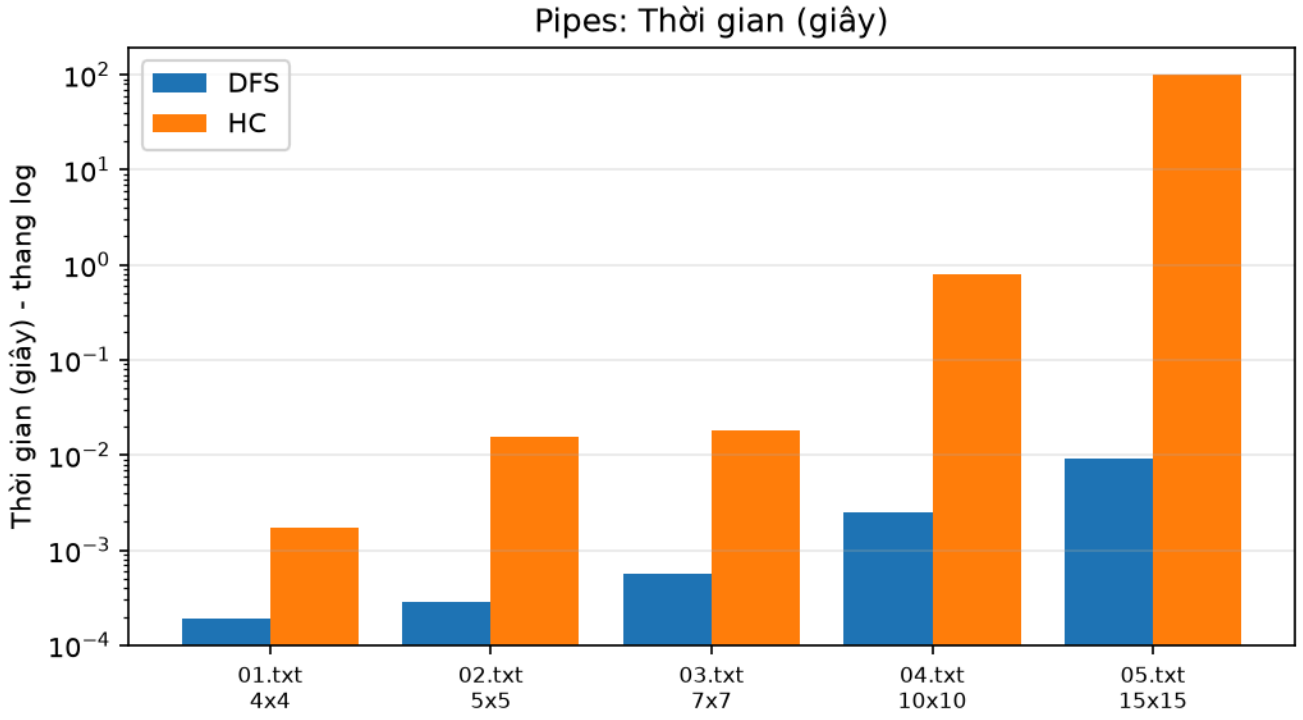
Bảy khó khăn thật đã gặp, theo thứ tự thời gian.

#	Vấn đề	Nguyên nhân	Cách giải quyết
1	Đề và lời giải chỉ có dạng ảnh chụp web, code không đọc được; chép tay đề 15×15 (225 ô) rất dễ sai	puzzle-pipes.com không cho tải đề dạng văn bản	Viết script đọc ảnh tự động (tìm lưới, xét mép từng ô xem có ống đi qua không), rồi kiểm tra chéo: lời giải phải là một cây khớp hết lỗ, mỗi ô đề phải là phép xoay của ô lời giải. Cả 5 cặp đều qua. Đề 4×4 được thay bằng đề có Puzzle ID để ghi được nguồn
2	Benchmark chạy nhầm 5 file lời giải như 5 đề mới	Benchmark đọc mọi file `inputs/pipes/*.txt`, mà ban đầu đề và lời giải nằm chung (`01_input.txt`, `01_solution.txt`)	Đổi tên đề thành `01.txt` … `05.txt` theo quy ước README, chuyển lời giải vào `inputs/pipes/solutions/`
3	Hill Climbing bản đầu (xoay 1 ô, xuất phát ngẫu nhiên, 20 restart) chỉ giải được đề 4×4	Nhiều trạng thái còn 2–4 đầu hở mà không lần xoay đơn nào làm h giảm (ridge); điểm xuất phát ngẫu nhiên nằm rất xa đích	Ba thay đổi: bỏ sẵn các góc chĩa ra biên khỏi miền; thêm láng giềng xoay 2 ô kề nhau quanh đầu hở; khởi tạo greedy theo ô trên/trái
4	Hill Climbing kẹt ở vùng bằng phẳng: không đi ngang thì chỉ giải được 3/5 đề dù cho 100 restart	Rất nhiều láng giềng có cùng h; Hill Climbing thuần dừng ngay khi không có láng giềng tốt hơn	Cho phép đi ngang tối đa 50 bước liên tiếp (tham số `sideways` trong engine của TV1). Kết quả: giải 5/5, đề 01–03 không cần restart
5	Đề 15×15 chạy rất chậm, timeout 300 s	Mỗi bước sinh vài nghìn láng giềng và tính lại h1 trên cả 225 ô cho từng cái	Tính h tăng dần (TV1 tối ưu): chỉ đếm lại quanh các ô vừa xoay, kết quả không đổi, nhanh hơn khoảng 5 lần. Đề 05 giải được trong khoảng 100 s
6	Ký hiệu ô không khớp với GUI	Ban đầu `encode()` trả về số thập phân (`12`, `10`) và `-1` cho ô chưa gán: lệch cột khi in, khác ký hiệu trong file đề, và GUI phải đoán quy ước	Thống nhất mã hex: `encode()` trả về ký tự `"0"` … `"F"` giống file đề, `"."` cho ô chưa gán. TV4 đọc bằng `int(c, 16)` trong `gui/render_pipes.py`

#	Vấn đề	Nguyên nhân	Cách giải quyết
7	`pipes.py` chưa chạy được với `main.py` và benchmark khi ghép với code TV1	Thiếu các phần registry bắt buộc	Thêm `load_problem(path)`, `name = "pipes"`, `size_label()`; thêm 4 test chạy với engine thật của TV1

9. Kết quả và nhận xét

Cả DFS và Hill Climbing đều giải được **5/5 đề**, nhưng DFS nhanh hơn từ khoảng 9 lần (4×4) đến khoảng 10.700 lần (15×15): đề 05 mất 0,0093 s với DFS và 99,7 s với Hill Climbing. DFS thắng vì ràng buộc khớp với ô trên/trái gần như ép chết góc xoay của mỗi ô, nên chỉ mở rộng 27–823 nút và giữ tối đa 46 nút trong stack. Hill Climbing phải sinh toàn bộ láng giềng ở mỗi bước: đề 05 sinh gần 17 triệu láng giềng qua 82 lần restart, nên bộ nhớ đỉnh cũng lớn hơn khoảng 11 lần (7.905 KB so với 688 KB). Kết quả đáng chú ý nhất nằm ở bảng phụ: bỏ đi ngang thì Hill Climbing chỉ giải được 3/5, đề 04 và 05 thất bại sau 100 restart với h tốt nhất còn 8 và 22. Ngay cả đề 7×7 cũng cần 63 restart, so với 0 restart khi có đi ngang, xác nhận vấn đề chính là vùng bằng phẳng.



Biểu đồ lấy nguyên từ `results/charts/pipes_time_s.png` (trục thời gian thang log). Các số còn lại:

Đề	Bộ nhớ DFS (KB)	Bộ nhớ HC (KB)	Nút mở rộng DFS	Nút mở rộng HC	Restart HC (đi ngang 50)	Restart HC (không đi ngang)
01 (4×4)	8,2	23,3	27	6	0	2
02 (5×5)	12,4	81,6	31	39	0	21
03 (7×7)	27,6	321,0	66	18	0	63
04 (10×10)	108,0	1.427,4	174	374	3	thất bại (100), h = 8
05 (15×15)	687,9	7.904,7	823	14.572	82	thất bại (100), h = 22

Nguồn: `results/benchmark.csv` (bảng chính, Hill Climbing 100 restart, đi ngang 50, seed 0) và `results/benchmark_variants.csv` (cột cuối), commit `d1390c8` "Ket qua benchmark chinh thuc" trên nhánh `main`. Với Hill Climbing, "nút mở rộng" là số trạng thái đã đứng và xét láng giềng. Hai dòng đi ngang 50 trong bảng phụ trùng số nút và restart với bảng chính, xác nhận seed cố định cho kết quả lặp lại.

10. Câu hỏi có thể bị hỏi

Hỏi 1: Vì sao DFS và Hill Climbing dùng hai cách hiểu trạng thái khác nhau? DFS gán dần từng ô nên không gian là một cây, không trạng thái nào lặp lại, và mỗi lần gán là cắt nhánh được ngay. Hill Climbing không lưu đường đi, chỉ so h giữa điểm hiện tại và các láng giềng, nên mỗi điểm phải là một lưới đầy đủ. Cả hai vẫn là tuple và dùng chung `goal_test`, `h`, `encode`.

Hỏi 2: Vì sao `is_valid` chỉ so với ô trên và ô trái? Vì ô được gán theo thứ tự hàng → cột, lúc gán ô (i, j) thì ô trên và ô trái chắc chắn đã có góc, còn ô dưới và ô phải chưa. Hai cạnh còn lại sẽ được kiểm tra khi tới lượt ô dưới và ô phải, nên mỗi cạnh được xét đúng một lần. Vòng và cụm kín cũng chỉ có thể xuất hiện quanh ô vừa gán, nên không cần quét lại cả lưới.

Hỏi 3: Vì sao Hill Climbing cần tới 82 lần restart ở đề 15×15 và chậm hơn DFS hàng nghìn lần? Hill Climbing là phương pháp cục bộ: nó chỉ nhìn hệ quả trước mắt nên hay kẹt ở cực tiểu địa phương và vùng bằng phẳng, càng nhiều khi lưới càng lớn. Bảng phụ cho thấy bỏ đi ngang thì đề 15×15 thất bại hẳn sau 100 restart. Pipes có ràng buộc chặt và cục bộ, nên phù hợp với tìm kiếm có hệ thống cộng cắt nhánh như DFS hơn là leo đồi.

Hỏi thêm: h1 có admissible không, và có cần không? Không: một lần xoay sửa được tới 8 đầu hở, nên h1 có thể lớn hơn số bước thật (đề 01: h1 = 16, chỉ cần đổi 13 ô). Admissible là điều kiện để A* tìm đường tối ưu; Hill Climbing không có g(n) và không đòi tối ưu, nên không cần.

11. Tài liệu tham khảo

1. Vương Bá Thịnh, *Problems and Search* (slide chương 2), Khoa KH&KT Máy tính, Trường ĐH Bách khoa – ĐHQG-HCM: mô hình không gian trạng thái, Depth-first search.
2. Vương Bá Thịnh, *Heuristic search* (slide chương 3), cùng nguồn: Steepest-Ascent Hill Climbing; local maximum, plateau, ridge; local và global heuristic (Block World); ký hiệu h(n), h*(n).
3. Đặc tả Bài tập lớn 1 (HK1 26-27), *Tìm kiếm* (`BTL1.pdf`).
4. puzzle-pipes.com: nguồn 5 đề (Puzzle ID 3,869,919; 8,335,297; 7,075,572; 723,397; 5,804,275), lấy ngày 24/10/2026.
5. Repo nhóm btl1-logic-puzzles, nhánh `main` , commit `d1390c8` : `puzzles/pipes.py` , `tests/test_pipes.py` , `core/search.py` , `core/check.py` , `results/benchmark.csv` , `results/benchmark_variants.csv` , `results/charts/pipes_time_s.png` , `gui/render_pipes.py` .
6. `puzzles/PIPES_NOTES.md` : ghi chú thiết kế và số liệu thử nghiệm của TV2.